Computational Study of Algorithms for the Parametric Maximum Closure Problem on Directed Forests

Technical Report

Valerio Dose Fabio Furini Marco Locatelli

August 2026

Abstract

This is a companion technical report to the manuscript *"On parametric Maximum Closure Problems over precedence forests"* (Dose, Furini and Locatelli). The manuscript introduces the algorithms studied here (`PaC`, `DPaC`, `HPaC`, `DHPaC`, `HIPaC`, `HOPaC` and `RaC`) together with their correctness proofs and complexity analysis; this report contains the full computational study – instance design, validation methodology, benchmark campaigns and results – at a level of detail appropriate for a technical report rather than for the manuscript itself. Section 1 recaps the problem and the algorithms compactly, for a reader without the manuscript at hand; Section 2 is the complete computational study.

1 Problem and algorithms recap

1.1 The parametric Maximum Closure Problem

Let $G = (V, A)$ be a directed acyclic graph on n items $V = \{1, \dots, n\}$, where arc $(i, j) \in A$ means that including item i forces the inclusion of item j. Each item i has an integer profit p_i (possibly negative) and a strictly positive integer weight w_i. For a closed subset S (i.e. $i \in S \Rightarrow j \in S$ for every $(i, j) \in A$), let $P(S) = \sum_{i \in S} p_i$ and $W(S) = \sum_{i \in S} w_i$. The parametric Maximum Closure Problem asks for the set of optimal solutions of $u(\lambda) := \max_{S \text{ closed}} P(S) - \lambda W(S)$ as a function of $\lambda \in \mathbb{R}$. Function u is convex and piecewise linear, with at most n breakpoints $\lambda_1 > \cdots > \lambda_k$; the solutions at consecutive breakpoints are nested, inducing a partition of V into k disjoint *closure layers* $\mathcal{I}_1, \dots, \mathcal{I}_k$, strictly decreasing in profit-to-weight ratio P_r/W_r. Computing this ordered partition – the *optimal sequence of closure layers* – is the computational task studied here. The manuscript restricts G to be a *directed forest* (the underlying undirected graph is a forest, arc orientation otherwise arbitrary) and gives algorithms specific to that case; this report benchmarks those algorithms empirically.

1.2 Algorithms benchmarked

`PaC` Peel-and-Contract: the manuscript's direct-scan reference algorithm for arbitrarily oriented forests, $O(n^2)$ worst-case time, $O(n)$ space; finds closure layers in decreasing ratio order by repeatedly peeling the current best final nodes or contracting an arc.

`DPaC` the dual variant of `PaC`, finding closure layers in increasing ratio order; same asymptotic complexity and scan-based, non-heap structure.

`HPaC` the heap-based practical variant of `PaC`: same worst-case complexity in general ($O(n^2 \log n)$), but replaces linear scans with a lazy heap, which the manuscript shows is faster in practice on most (not all) topologies.

`DHPaC` the dual of `HPaC`.

`HIPaC / HOPaC`
specialized heap-based variants for forests known in advance to be entirely in-trees (out-degree ≤ 1 everywhere) or out-trees (in-degree ≤ 1 everywhere) respectively; $O(n \log n)$ worst-case time in both cases.

`RaC` Rake-and-Compress: a top-tree-based algorithm with $O(n \log n)$ time and space, derived in the manuscript following suggestions from `Claude Fable 5`; applicable to general trees regardless of orientation.

`BPPF` Bounded-Precision Parametric Pseudoflow, the unmodified upstream implementation of Hochbaum's parametric pseudoflow method for general precedence graphs (not restricted to forests); used here purely as a third-party, independent correctness oracle, and at a scope-limited size as a general parametric-flow reference point.

Every algorithm returns the same canonical object – the ordered sequence of closure layers with their exact rational thresholds – computed with exact integer/rational arithmetic throughout (no floating point in any decision path). Full derivations, pseudocode and complexity proofs for `PaC`/`DPaC`/`HPaC`/`DHPaC`/`HIPaC`/`HOPaC` and `RaC` are in the manuscript; this report takes them as given and evaluates them empirically.

---

2 Computational Results

2.1 Instance generation

Every instance is stored in a small, capacity-free text format: a forest on n items given by its arc list, one integer profit p_i (possibly negative) and one strictly positive integer weight w_i per item i, where arc (u, v) encodes the closure implication $x_u \leq x_v$. No capacity field exists anywhere in this format and no reader is allowed to accept one. Every instance file is produced by a seeded, deterministic Python generator – never hand-edited – and is fully determined by four independent choices: a *topology* (Table 1), an *affine-coefficient family* (Table 2), a size n, and a random seed.

The first four families give, respectively: dispersed positive ratios (`independent-positive`); the same but with some items locally unfavorable, since p_i may be negative (`independent-signed`); ratios concentrated near 1 (`correlated`); and more heterogeneous ratios obtained by anti-correlating p_i with w_i (`anti-correlated`). The last two stress exact rational comparisons directly: for `near-ties`, a target ratio num/den is drawn once per instance (num, den uniform on $\{1, \ldots, 20\}$); for every item i, t_i is uniform on $\{1, \ldots, 50\}$, $w_i = \text{den} \cdot t_i$ and $p_i = \text{num} \cdot t_i + \jmath_i$ with jitter $\jmath_i$ uniform on $\{-2, -1, 1, 2\}$, so many items share a common ratio up to a small integer perturbation. For `exact-ties`, the n items are split into $\max(1, \lfloor n/20 \rfloor)$ groups, each assigned its own exact ratio $\text{num}_g/\text{den}_g$ (drawn uniformly on $\{1, \ldots, 20\}^2$); for an item i in group g, t_i is uniform on $\{1, \ldots, 50\}$, $w_i = \text{den}_g \cdot t_i$ and $p_i = \text{num}_g \cdot t_i$, so every item in a group shares the exact same ratio $p_i/w_i = \text{num}_g/\text{den}_g$ – stressing the canonicalization rule that merges consecutive closure layers at an equal threshold.

Topology	Construction rule	Degree/shape
`mixed-forest`	connect each item $v \in \{2,\dots,n\}$ to a uniformly chosen predecessor $u < v$, independently with probability ρ; orient every resulting edge as (u,v) or (v,u), each with probability $1/2$	general forest
`in-forest`	scan $u \in \{1,\dots,n-1\}$; independently with probability ρ, add one arc (u,j) with j uniform on $\{u+1,\dots,n\}$	out-degree ≤ 1
`out-forest`	scan $v \in \{2,\dots,n\}$; independently with probability ρ, add one arc (u,v) with u uniform on $\{1,\dots,v-1\}$	in-degree ≤ 1
`path-mixed`	fixed path shape (item v's parent is item $v-1$); orient every edge independently, each with probability $1/2$	path, no ρ
`binary-mixed`	fixed balanced binary tree (item v's parent is item $\lfloor (v-1)/2 \rfloor$); orient every edge independently, each with probability $1/2$	binary tree, no ρ
`star-mixed`	fixed star (every item $v > 0$'s parent is item 0); orient every edge independently, each with probability $1/2$	star, no ρ

Table 1: The six topology families. Density $\rho \in \{0.3, 0.6, 0.9, 1.0\}$ (`sparse` to `conn`) applies only to the three random topologies; the three structured families are always a complete tree on n items, with only the arc orientations randomized.

Benchmark campaigns. Every combination of topology, coefficient family, size and seed used in this section belongs to one of six named campaigns, frozen in `docs/EXPERIMENTAL_PROTOCOL.md` before any official run; Table 3 summarizes all six, and every result subsection below states which campaign it draws on. Every campaign draws from all six coefficient families of Table 2.

Manifests and reproducibility. Every generated instance is manifested with its SHA-256 checksum, topology classification and coefficient bounds (`instances/manifests/`), so that provenance and integrity can be checked without regenerating the instance. The exact generation rule for every topology and every affine-coefficient family, including the underlying pseudo-random routines, is given above.

2.2 Experimental goals and research questions

The computational study answers six questions about algorithms for the parametric maximum-closure problem on directed forests, each with its own subsection below:

1. Is the heap-based scan (`HPaC`) actually preferable to the direct-scan reference algorithm (`PaC`), and are there structured cases where the opposite holds? (Section 2.6)
2. How does `HPaC` scale empirically on directed forests with arbitrary orientation? (Section 2.7)
3. How much do the specialized heap variants for in-forests (`HIPaC`) and out-forests (`HOPaC`) gain over the general `HPaC`/`DHPaC` pair when the orientation is known in advance? (Section 2.8)

Coefficient family	w_i	p_i
`independent-positive`	$\mathrm{U}\{1,\dots,1000\}$	$\mathrm{U}\{1,\dots,1000\}$, indep.
`independent-signed`	$\mathrm{U}\{1,\dots,1000\}$	$\mathrm{U}\{-1000,\dots,1000\}$, indep.
`correlated`	$\mathrm{U}\{1,\dots,1000\}$	$w_i + \delta_i$
`anti-correlated`	$\mathrm{U}\{1,\dots,1000\}$	$(1001 - w_i) + \delta_i$
`near-ties`	$\mathrm{den} \cdot t_i$	$\mathrm{num} \cdot t_i + \jmath_i$
`exact-ties`	$\mathrm{den}_g \cdot t_i$	$\mathrm{num}_g \cdot t_i$

Table 2: The six affine-coefficient families, chosen only for the shape of the resulting ratio p_i/w_i, never for any external classification; $\delta_i \sim \mathrm{U}\{-50,\dots,50\}$ throughout. `near-ties` and `exact-ties` are defined in full in the text below.

Campaign	Topologies	Sizes n	ρ	Seeds	Instances	Algorithms
A – small correctness	all six	≤ 10 (11 random; exhaustive at 4)	all	–	–	oracle + every algorithm
B – medium random	`mixed-forest`	100–1 000 (step 100)	0.3–1.0	10	2 400	`PaC`, `HPaC`, `RaC`
C – large random	`mixed-forest`	10 000–100 000 (step 10 000)	0.3–1.0	10	2 400	`HPaC`, `RaC` (`PaC` at n=10 000)
D – structured stress	`path/binary/star-mixed`	100–100 000 (10 sizes)	–	10	600/topology	`HPaC` vs. `RaC` (`PaC` at $n \leq 2\,000$)
E – specialized orientations	`in-/out-forest`	100–1 000 $\cup$ 10 000–100 000	0.6, 1.0	5	1 200/orientation	`HPaC`+`HIPaC`/`HOPaC`+`RaC`
F – `BPPF` baseline	`mixed-forest`	100, 200, 500, 1 000	0.6	3	72	`HPaC`, `RaC`, `BPPF`

Table 3: The six benchmark campaigns. Campaign D's instance counts are per structured topology (1 800 total); campaign E's are per orientation (2 400 total). Campaign A validates correctness (Section 2.4) and contributes no timing numbers.

4. How do `HPaC` and the rake-and-compress algorithm (`RaC`) compare on random forests? (Section 2.10)

5. Which structural families favor `RaC` and which favor `HPaC`? (Section 2.11)

6. How do these forest-specific algorithms compare with a general parametric-flow method not restricted to forests? (Section 2.9)

Every subsection below is backed by a complete campaign with a preregistered instance matrix, or is explicitly marked exploratory where it is not.

2.3 Algorithms and implementations

All algorithms compared here are implemented natively for the closure model of the accompanying manuscript's forest-closure model (integral profit p_i, strictly positive integral weight w_i, affine node cost $p_i - \lambda w_i$, arc (u, v) meaning $x_u \leq x_v$): there is no capacity, no relaxed linear program and no reconstruction of a knapsack solution anywhere in the implementation.

- `PaC`/`DPaC`: the direct-scan reference implementation of the manuscript's main forest algorithm and its dual, for arbitrarily oriented forests. `DPaC` shares `PaC`'s scan-based, non-heap structure and its empirical $O(n^2)$-like scaling (Section 2.7).

- `HPaC`/`DHPaC`: the heap-based practical variant and its dual, for arbitrarily oriented forests. Both are exercised throughout Sections 2.6, 2.7, 2.10 and 2.11. `DHPaC` is consistently slower than `HPaC` on mixed-orientation random forests – median `DHPaC`/`HPaC` is $\approx$ 1.4–1.55 on the medium campaign (B) and $\approx$ 1.6–1.7 on the large campaign (C), stable across sizes – but the two remain within the same order of magnitude and neither changes which algorithm

family (heap-based general vs. `RaC`) is preferable; `DHPaC` is reported alongside `HPaC` rather than in a separate subsection for this reason, not because the two are numerically close.

- `HIPaC`/`HOPaC`: the specialized heap-based variants for in-forests and out-forests, respectively.
- `RaC`: the rake-and-compress/top-tree implementation of the algorithm described in the accompanying manuscript, ported from an experimental package and audited operation-by-operation against that source and against the manuscript (`docs/RAC_AUDIT.md`).
- `BPPF`: the unmodified upstream bounded-precision parametric pseudoflow implementation, used both as an independent verification oracle (Section 2.4) and, at a scope-limited size, as a general parametric-flow baseline (Section 2.9).

Every algorithm returns the same canonical object: the ordered sequence of closure layers, their exact rational thresholds, and the induced closure at every breakpoint. All rational comparisons use exact signed 64-bit numerators and denominators compared via signed 128-bit cross-multiplication; no floating-point value appears anywhere in an algorithm's decision path.

2.4 Validation methodology

Correctness is established independently of any single algorithm and before any timing claim:

- **Exhaustive enumeration.** For every directed forest with at most four items over a finite affine-coefficient grid, and for thousands of random forests up to eleven items, every algorithm's output is checked against an independent oracle that enumerates all closed sets, builds their affine lines and computes the exact upper envelope.
- **Maximum closure at a fixed λ, via an independent max-flow engine.** For a chosen rational λ, the instance is reduced to the standard maximum-weight-closure network and solved with `BPPF`, a third-party pseudoflow implementation unrelated to any algorithm above; agreement with our own algorithms' closure at that λ was checked at every breakpoint midpoint across random, path and star topologies and all six affine-coefficient families, with zero disagreements before `BPPF` was trusted for Sections 2.4 and 2.9.
- **Cross-algorithm differential agreement.** Every official campaign benchmarks multiple algorithms on the same instance and compares their returned sequences (partition, thresholds and canonical order, not only the closure layer count); any disagreement is reported explicitly rather than averaged away.

2.5 Experimental environment and protocol

All algorithms are implemented in C++ and run single-threaded. Experiments were run on a Linux machine (Ubuntu 22.04.5, kernel 6.8) equipped with an Intel Core i7-12700 processor and 32 GB of RAM, GCC 11.4.0 at `-O3`. Every benchmark process is pinned to one physical core; algorithm order is independently shuffled per repetition so no algorithm is systematically favored by CPU frequency state. Only the algorithm call itself is timed: instance parsing, correctness verification and result serialization are excluded from every reported time. Headline comparisons are always paired per instance (the same instance, same process-invocation window, for every algorithm compared), and reported as the median and interquartile range of the per-instance

Campaign	Instances	Mismatches
campaign_b	2400	0
campaign_c	2400	0
campaign_d_binary	600	0
campaign_d_path	600	0
campaign_d_star	600	0
campaign_e_in	1200	0
campaign_e_out	1200	0

Table 4: Instances and cross-algorithm disagreements per campaign, out of every instance generated. Campaign D-star's 600 instances include the 180 on which `HPaC`/`DHPaC` hit the memory ceiling (Section 2.11); those are excluded from the disagreement count shown here, since a censored run produces no sequence to compare.

ratio, not a ratio of aggregate means. A preregistered wall-clock timeout (300 s) and a memory ceiling (8 GB, enforced with `ulimit -v`) apply to every run; an instance that hits either is recorded as a censored observation, never silently dropped or estimated. The machine's CPU governor could not be set to `performance` (no passwordless root); this is a documented source of noise in absolute wall-clock numbers, mitigated by reporting paired ratios rather than raw times (`docs/EXPERIMENTAL_PROTOCOL.md`, "Known measurement limitation").

All numbers below are computed by `tools/build_report.sh` directly from `results/raw/*.csv` (raw) and `results/processed/` (paired medians and ratios); none is hand-typed. Every campaign in Section 2.1 was run to completion (or to its preregistered censoring point) exactly as specified in `docs/EXPERIMENTAL_PROTOCOL.md`. Across all 9 000 benchmarked instances and 203 880 timed runs spanning campaigns B, C, D and E, **every algorithm pair benchmarked on the same instance returned the identical closure layer sequence**: zero disagreements (Table 4).

2.6 PaC versus HPaC

On mixed-orientation random forests (campaign B, $n \in \{100, \ldots, 1000\}$, 2 400 instances), `HPaC` and `PaC` are comparable at small n and `HPaC` pulls ahead only as n grows, consistently with the manuscript's theoretical prediction that a heap cannot improve `HPaC`'s worst-case complexity but should help in practice: median `PaC`/`HPaC` is 0.78 at $n = 100$ (`PaC` is in fact marginally faster here, where heap-maintenance overhead outweighs its benefit), crosses 1 around $n \approx 400$, and reaches 1.72× at $n = 1\,000$ (`campaign_b_pac_over_hpac.tex`), consistent with `PaC`'s scan-based construction paying a growing cost per breakpoint as n increases while `HPaC`'s heap does not. `DPaC` tracks `PaC` closely, as expected from sharing its scan structure, and opens no separate storyline.

The countercase anticipated by the manuscript is confirmed quantitatively on mixed-orientation stars (campaign D, PaC-eligible subset, $n \in \{100, \ldots, 2000\}$): `HPaC` is *dramatically slower* than `PaC` at every size tested (Table 5), by a median factor growing from 17.2× at $n = 100$ to 48.4× at $n = 2\,000$. The operational reason is structural, not incidental: every update to the central node's cumulative closure sum invalidates the candidate ratio of a large fraction of the remaining leaves at once, and `HPaC`'s lazy heap must absorb that many stale entries before it can pop a valid minimum again, while `PaC`'s direct scan pays no such maintenance cost. This same mechanism is responsible for the far larger effect at scale described in Section 2.11: on stars with $n \geq 20\,000$,

n_nodes	Paired instances	Median hpac/pac	IQR
100	60	17.220	1.113
200	60	25.872	1.670
500	60	36.641	5.338
1000	60	40.175	7.588
2000	60	48.385	10.176

Table 5: Mixed-star, median `HPaC/PaC` paired ratio (campaign D, PaC-eligible subset). Values above 1 favor `PaC`.

`HPaC` does not merely lose to `PaC`, it exhausts the 8 GB memory ceiling entirely.

2.7 HPaC scaling on arbitrarily oriented forests

On the large random campaign (C, $n \in \{10\,000, \ldots, 100\,000\}$, mixed-forest, 2 400 instances), `HPaC` stays fast in absolute terms: median elapsed time grows from a few hundred microseconds at $n = 10\,000$ to under a second at $n = 100\,000$ in the same runs used for Table 8, with no instance approaching the 300 s timeout. `DHPaC`, the dual general heap-based algorithm, is consistently slower than `HPaC` on this campaign – median `DHPaC/HPaC` ≈ 1.6–$1.7\times$, stable across every size from $n = 10\,000$ to $n = 100\,000$ (`campaign_c_dhpac_over_hpac.tex`), close to the ≈ 1.4–$1.55\times$ already observed on the medium campaign B. The two never swap places and neither changes which algorithm family (heap-based general vs. `RaC`) is preferable; the gap is attributed to scan-direction bookkeeping, not to a qualitatively different algorithm.

2.8 Specialized orientations

Campaign E ($n \in \{100, \ldots, 1000\} \cup \{10\,000, \ldots, 100\,000\}$, $\rho \in \{0.6, 1.0\}$, 5 seeds, 1 200 instances per orientation) compares the specialized heap variants against the general algorithm on the orientation each is built for. `HIPaC` is faster than `HPaC` on in-forests at every size, with a median ratio `HIPaC/HPaC` between 0.27 and 0.50 (i.e. `HIPaC` takes roughly a third to a half of `HPaC`'s time), and `HOPaC` is faster than `HPaC` on out-forests with a median ratio between 0.35 and 0.59 (Table 6). Both specializations retain their advantage across the entire size range tested, including $n = 100\,000$, with no sign of the gain eroding at scale.

On the same in-forest instances, `RaC` loses ground to `HPaC` as n grows: median `RaC/HPaC` rises from ≈ 1.9–$2.8\times$ on the medium range to $8.5\times$ at $n = 100\,000$ (`campaign_e_in_rac_over_hpac.tex`), mirroring the trend already seen on general mixed forests in Section 2.10. The out-forest picture (`campaign_e_out_rac_over_hpac.tex`) is qualitatively the same.

2.9 General parametric-flow baseline

Campaign F pairs `HPaC` and `RaC` against `BPPF`, an unmodified upstream bounded-precision pseudoflow solver, on $n \in \{100, 200, 500, 1000\}$ mixed-forest instances across all six coefficient families (72 instances). Because the `.pcf`-to-`BPPF` converter bakes one exact rational λ into integer arc capacities per call (Section 2.4), `BPPF` is driven once per breakpoint `HPaC` reports rather than swept natively; agreement is exact on every one of the 72 instances (**zero mismatches** between `BPPF`'s minimum-cut closure and `HPaC`'s at every sampled breakpoint midpoint). The resulting total time – summed over one-process-per-breakpoint invocations – is between $91\times$ and $1\,270\times$

n_nodes	Paired instances	Median hipac/hpac	IQR
100	60	0.496	0.427
200	60	0.425	0.427
300	60	0.437	0.442
400	60	0.417	0.482
500	60	0.359	0.511
600	60	0.327	0.451
700	60	0.364	0.462
800	60	0.348	0.491
900	60	0.326	0.492
1000	60	0.321	0.528
10000	60	0.275	0.483
20000	60	0.284	0.507
30000	60	0.289	0.497
40000	60	0.306	0.520
50000	60	0.289	0.545
60000	60	0.288	0.532
70000	60	0.296	0.554
80000	60	0.305	0.545
90000	60	0.315	0.564
100000	60	0.304	0.564

Table 6: In-forest, median `HIPaC`/`HPaC` paired ratio (campaign E). Values below 1 favor `HIPaC`.

`HPaC`'s in-process time on the same instance; this number is process-spawn-dominated by construction and is reported only as evidence that `BPPF` is a correct, independent oracle at this scale, never as a native-speed comparison with the in-process forest algorithms.

2.10 HPaC versus RaC on random forests

On the medium campaign (B), the median paired ratio `RaC`/`HPaC` is stable between 2.6× and 3.0× across every size from $n = 100$ to $n = 1\,000$ (Table 7), with no strong dependence on density or coefficient family. On the large campaign (C), the same ratio grows steadily with n: from 3.1× at $n = 10\,000$ to 12.9× at $n = 100\,000$ (Table 8), with an increasingly wide interquartile range at the largest sizes, meaning `RaC`'s relative disadvantage becomes both larger and more variable as forests grow. `DHPaC`/`RaC` shows the same qualitative trend as `HPaC`/`RaC` at every size tested; the primal/dual choice never changes which algorithm family is preferable on random forests. Every instance in both campaigns completed well inside the timeout, with zero timeouts and zero memory censoring.

2.11 Structured-tree stress tests

Path and binary trees are, like random forests, favorable to `HPaC`: median `RaC`/`HPaC` rises from 5.1× at $n = 100$ to 7.2–7.3× at $n \geq 10\,000$ on paths, and stays close to 2.4–2.8× across the whole range on balanced binary trees (Tables 9 and 10). Every path and binary instance completed within the timeout with no censoring.

n_nodes	Paired instances	Median rac/hpac	IQR
100	240	2.949	0.749
200	240	2.879	0.737
300	240	2.832	0.779
400	240	2.708	0.876
500	240	2.684	1.134
600	240	2.748	1.087
700	240	2.685	1.043
800	240	2.690	1.168
900	240	2.727	1.270
1000	240	2.593	1.226

Table 7: Mixed-forest medium campaign (B), median `RaC`/`HPaC` paired ratio by size. Values above 1 favor `HPaC`.

n_nodes	Paired instances	Median rac/hpac	IQR
10000	240	3.126	2.839
20000	240	3.075	4.346
30000	240	4.703	6.389
40000	240	6.293	8.261
50000	240	6.989	10.515
60000	240	8.402	13.110
70000	240	9.195	15.068
80000	240	10.762	17.332
90000	240	12.277	19.701
100000	240	12.858	22.093

Table 8: Mixed-forest large campaign (C), median `RaC`/`HPaC` paired ratio by size. Values above 1 favor `HPaC`.

Mixed-orientation stars invert this picture completely, and do so increasingly sharply as n grows. On the sizes where both algorithms survive ($n \leq 10\,000$, Table 11), median `RaC`/`HPaC` falls from 0.424 at $n = 100$ to 0.002 at $n = 10\,000$: `RaC` is already 2.4× faster than `HPaC` in the median at $n = 100$, and roughly 500× faster at $n = 10\,000$. Beyond that size the comparison becomes one-sided in a stronger sense: *every one* of the 60 star instances at $n = 20\,000$, every one at $n = 50\,000$, and every one at $n = 100\,000$ made both `HPaC` and `DHPaC` exceed the 8 GB memory ceiling (180 censored instances in total, recorded as `status=oom` in the raw data, never estimated or imputed), while `RaC` completed all 180 of the same instances comfortably, with a median time of 0.593 s at $n = 100\,000$ and no failures. A single-repetition, single-algorithm reference measurement (Release build, same machine) makes the memory picture concrete: at $n = 100\,000$, `HPaC` throws `std::bad_alloc` after allocating on the order of 25 GB, whereas `RaC` peaks at 213 MiB on the same instance. `RaC`'s own scaling on stars is close to linear across three orders of magnitude, from 0.541 ms at $n = 100$ to 592 ms at $n = 100\,000$ (Table 12).

The operational explanation mirrors Section 2.6: every update to the star's central node invalidates a large fraction of the remaining leaves' candidate ratios at once, so `HPaC`'s lazy heap accumulates stale entries at a rate that grows with n itself, not just with the number of genuine

n_nodes	Paired instances	Median rac/hpac	IQR
100	60	5.087	0.403
200	60	5.431	0.332
500	60	5.735	0.503
1000	60	6.050	0.560
2000	60	6.338	0.767
5000	60	6.855	0.869
10000	60	7.097	0.911
20000	60	7.250	1.076
50000	60	7.338	1.301
100000	60	7.236	1.404

Table 9: Path-mixed, median `RaC`/`HPaC` paired ratio by size (campaign D). Values above 1 favor `HPaC`.

n_nodes	Paired instances	Median rac/hpac	IQR
100	60	2.813	0.586
200	60	2.749	0.395
500	60	2.721	0.412
1000	60	2.677	0.242
2000	60	2.683	0.378
5000	60	2.678	0.367
10000	60	2.663	0.331
20000	60	2.651	0.398
50000	60	2.535	0.430
100000	60	2.414	0.429

Table 10: Binary-mixed, median `RaC`/`HPaC` paired ratio by size (campaign D). Values above 1 favor `HPaC`.

breakpoints; `RaC`'s rake operations absorb the star's leaves in a small number of rounds regardless of the central node's update history, which is exactly the structural case rake-and-compress is built for.

2.12 Computational conclusions

Correctness is uniform across the entire study: 9 000 benchmarked instances, 203 880 timed runs, seven algorithms, zero disagreements, plus 481 independent breakpoint-midpoint cross-checks against `BPPF` (Section 2.4) and exact agreement on every one of the 72 campaign-F instances (Section 2.9). No algorithm in this study returned an incorrect sequence on any instance we generated.

`HPaC` (and its dual `DHPaC`, consistently 1.4–1.7× slower but never changing the qualitative picture) is the preferred general algorithm on mixed-orientation random forests, paths and balanced binary trees, with its advantage over `RaC` growing from a stable 2–3× at medium size to over 12× at $n = 100\,000$ on random forests. `RaC` is the preferred algorithm on mixed-orientation stars, where its advantage over `HPaC` also grows with n – from $\approx 2\times$ at $n = 100$ to $\approx 500\times$

n_nodes	Paired instances	Median rac/hpac	IQR
100	60	0.424	0.022
200	60	0.202	0.010
500	60	0.076	0.009
1000	60	0.036	0.005
2000	60	0.016	0.003
5000	60	0.005	0.001
10000	60	0.002	0.000

Table 11: Mixed-star, median `RaC`/`HPaC` paired ratio, sizes where both algorithms survived the memory ceiling (campaign D). Values below 1 favor `RaC`; note the ratio falls by two orders of magnitude across this range.

n	instances	median `RaC` time
100	60	0.541 ms
200	60	1.167 ms
500	60	3.052 ms
1 000	60	6.174 ms
2 000	60	13.060 ms
5 000	60	33.959 ms
10 000	60	52.629 ms
20 000	60	107.616 ms
50 000	60	283.955 ms
100 000	60	592.398 ms

Table 12: `RaC`'s own median time on mixed-star instances across the full campaign D size range, including the sizes at which `HPaC` and `DHPaC` are entirely censored by the memory ceiling.

at $n = 10\,000$ – and becomes categorical beyond $n = 20\,000$, where `HPaC` is not merely slower but exhausts an 8 GB memory ceiling on every instance tested while `RaC` continues to scale near-linearly with a memory footprint two orders of magnitude smaller. `HIPaC` and `HOPaC` deliver a stable 2–3× speedup over the general `HPaC` on in-forests and out-forests respectively, at every size tested up to $n = 100\,000$, with no sign of the gain closing at scale. `BPPF`, used here purely as an independent verification oracle and a correctness-scoped baseline, agrees with every algorithm in this study on every instance tested; the domains are different by construction (general graphs vs. forests), so no relative performance claim between `BPPF` and the forest-specific algorithms is made beyond the explicitly process-spawn-dominated numbers in Section 2.9. No algorithm in this study is described as "state of the art": each claim above is scoped to the topology, size range and coefficient families in which it was actually measured.

Code and Data Availability

The code, instance generators, raw and processed benchmark data, and the analysis pipeline for this report are available at https://github.com/fabiofurini/parametric-closure-forests, release `v0.2.0` (commit `59f80f5`), which is independent of and does not depend on the repository

accompanying the precedence-constrained-knapsack paper. The release attaches the full instance archive (`instances.zip`) and raw and processed benchmark data (`results.zip`) with SHA-256 checksums. The repository is public. The pseudoflow oracle used for independent validation is the unmodified upstream Bounded-Precision Parametric Pseudoflow implementation available at https://github.com/hochbaumGroup/Bounded-precision-simple-parametric.git.